

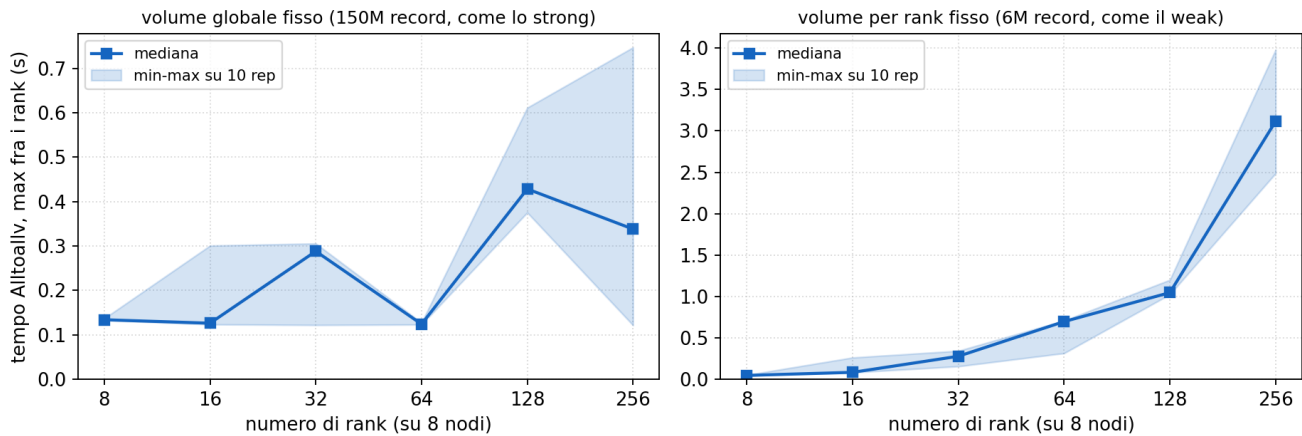
Modulo 4: esperimenti aggiuntivi

Esperimenti a supporto del report del Modulo 4 (partitioned hash join distribuito, MPI e MPI+OpenMP). Per ciascuno il grafico e i punti principali. Misure sui nodi Ivy Bridge del cluster (Xeon E5-2640 v2, 32 CPU logiche, 2 nodi NUMA per nodo), gli stessi del report, con job SLURM esclusivi fino a 8 nodi. Parametri del report dove applicabile: NR=50M, NS=100M, P=256, seed 42, max_key=25M.

Esperimento	Riferimento nel report
Esp. 1: anatomia di MPI_Alltoallv	Sez. 4.2, il dip a 128 rank e la varianza
Esp. 2: modello di Hockney misurato	Sez. 4.3, weak scaling e termine di startup
Esp. 3: continuum rank per nodo	Sez. 4-5, pure MPI vs hybrid
Esp. 4: imbalance e remapping sotto skew	Sez. 4.5, hard floor e partition-aware remapping
Esp. 5: sensibilità a P	Sez. 4.1, scelta P=256
Esp. 6: livello di thread support	Sez. 6, FUNNELED vs MULTIPLE

Esp. 1: anatomia di MPI_Alltoallv

Alltoallv sintetico: il costo segue il rank count, non i nodi

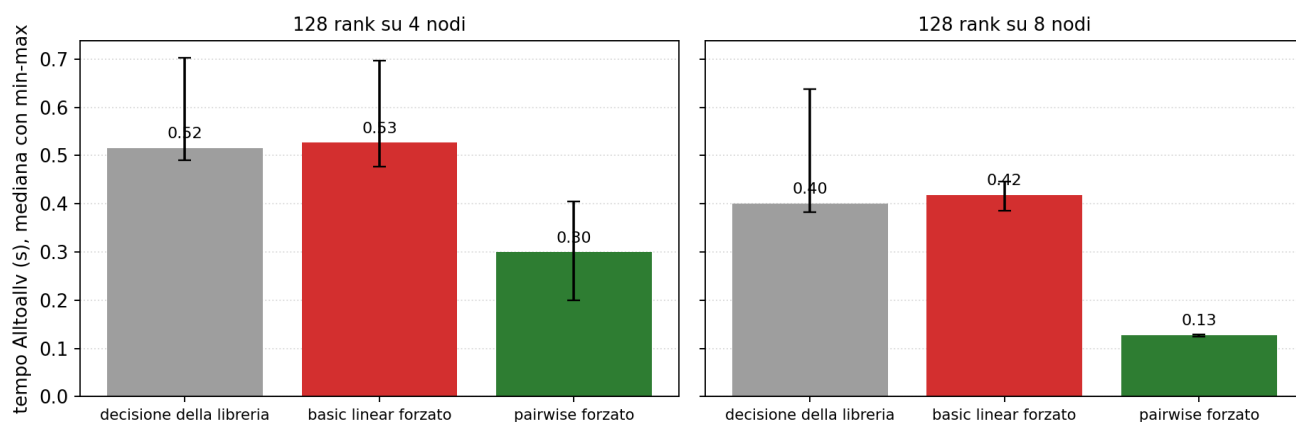


Sweep del rank count con buffer sintetici, fuori dalla pipeline.

- Il microbenchmark ripete il solo scambio Alltoallv della pipeline con buffer sintetici (barrier, timer, riduzione max; 10 ripetizioni, banda min-max nel grafico).
- A volume globale fisso (pannello sinistro, la forma dello strong scaling) il costo non è monotono nel rank count: 64 rank su 8 nodi scambiano in 0.12 s, 128 rank in 0.43 s con metà dei dati per rank; a 256 rank la mediana scende ma la dispersione esplode (0.12-0.75 s).
- Il dip del report a 4 nodi / 128 rank è quindi una proprietà della collettiva a quel rank count, riprodotta qui senza join, senza generatore e su un set di nodi fisso.
- A volume per rank fisso (pannello destro, la forma del weak scaling) il costo cresce di 69 volte da 8 a 256 rank: la controparte sintetica dei 0.12 s -> 3.17 s della pipeline.

Esp. 1: anatomia di MPI_Alltoallv

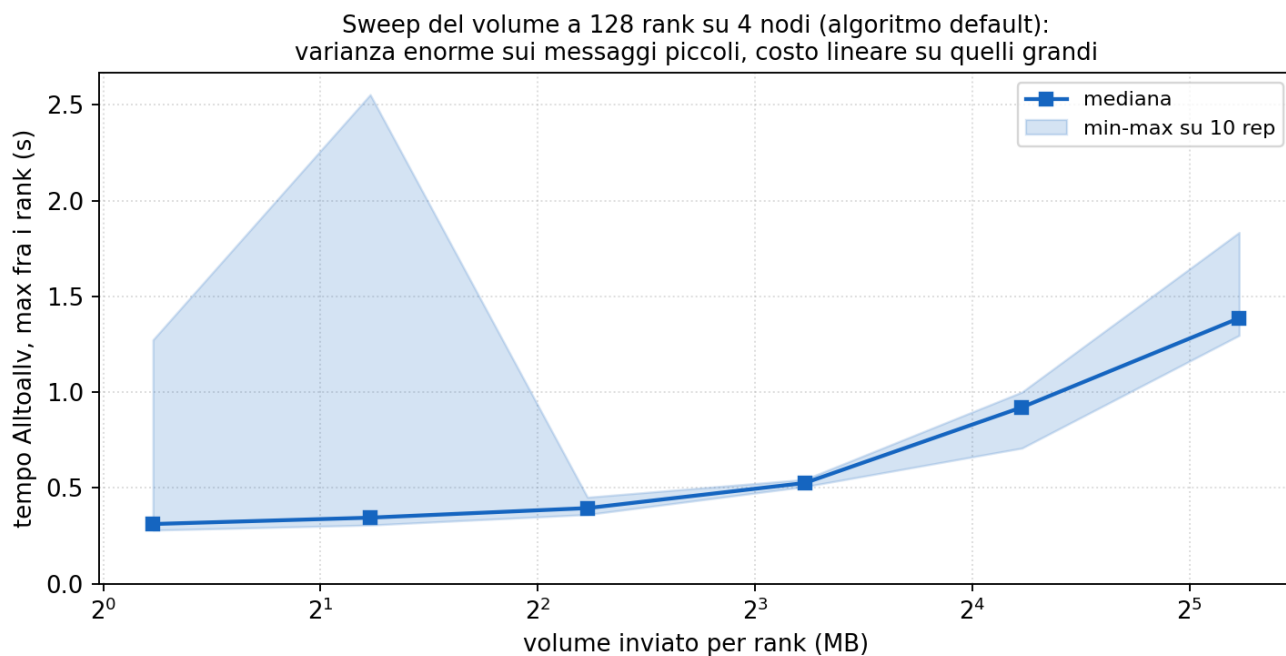
Algoritmo della collettiva forzato via MCA (stesso volume del caso strong)



Algoritmo della collettiva forzato via parametri MCA.

- Il report mostra che forzare basic linear rende lo scambio stabile ma uniformemente lento; qui il confronto include pairwise, l'altro algoritmo di coll tuned per Alltoallv.
- Pairwise forzato batte la decisione della libreria di un fattore 1.7 a 4 nodi (0.30 contro 0.52 s) e di un fattore 3.1 a 8 nodi (0.13 contro 0.40 s), con varianza quasi nulla a 8 nodi.
- La decisione automatica della libreria in questo regime coincide di fatto con basic linear (0.52 e 0.53 s a 4 nodi) ed è quella sbagliata: un singolo parametro MCA (`coll_tuned_alltoallv_algorithm=2`) avrebbe rimosso gran parte del dip osservato nello strong scaling.

Esp. 1: anatomia di MPI_Alltoallv

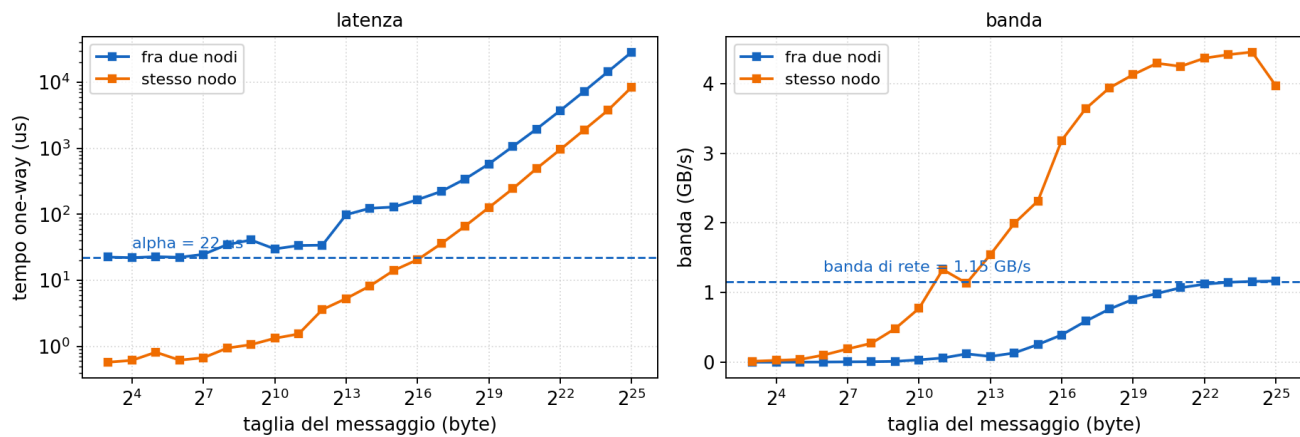


Sweep del volume per rank a 128 rank su 4 nodi.

- A parità di 128 rank il costo mediano cresce con il volume (0.31 -> 1.39 s per 1.2 -> 37.5 MB per rank), ma in modo molto meno che proporzionale nella parte bassa: il costo fisso della sincronizzazione a 128 partecipanti domina i messaggi piccoli.
- La varianza run-to-run è massima proprio sui volumi piccoli (min-max da 0.15 a oltre 2 s a 2.3 MB per rank): il regime instabile osservato nel report vive dove i messaggi per coppia sono piccoli e il fan-out è alto.

Esp. 2: modello di Hockney misurato

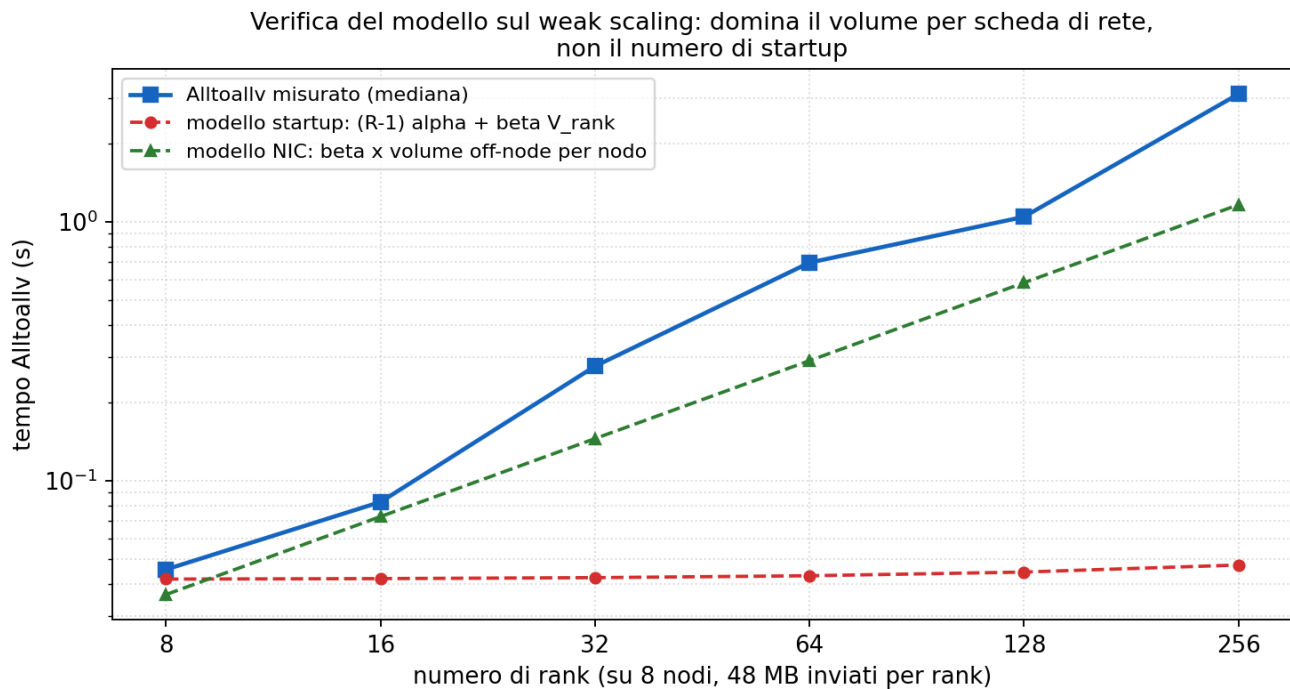
Ping-pong: i coefficienti del modello di Hockney misurati sul cluster



Ping-pong fra due rank: latenza e banda, inter e intra nodo.

- Il report usa il modello di Hockney ($\alpha + \beta m$) senza coefficienti misurati; il ping-pong li fornisce: $\alpha = 22$ us e banda asintotica 1.15 GB/s fra nodi diversi (compatibile con 10 Gb Ethernet), 0.6 us e 4.3 GB/s dentro il nodo.
- Il rapporto fra le due bande (circa 4x) e fra le due latenze (circa 37x) quantifica quanto lo scambio intra-nodo sia più economico: è la ragione per cui conta soprattutto il volume che attraversa la scheda di rete.

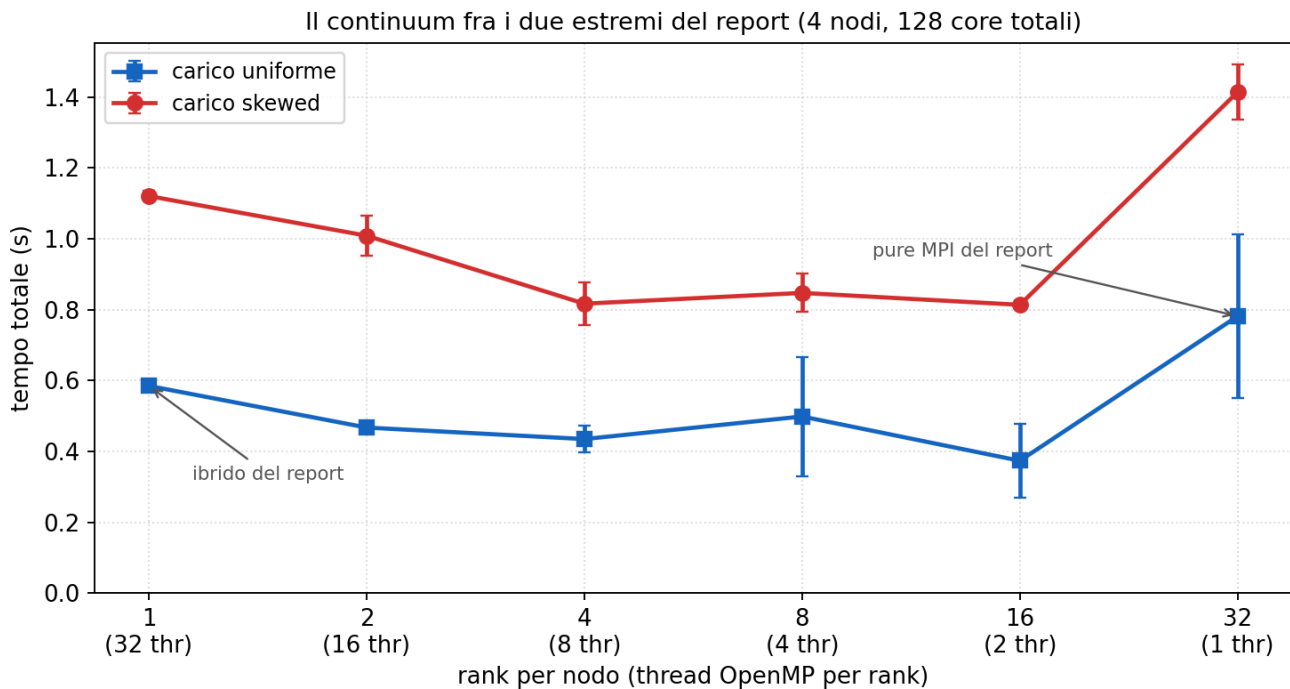
Esp. 2: modello di Hockney misurato



Il weak scaling dell'Alltoallv contro i due modelli.

- Con i coefficienti misurati, il termine di startup del report vale $(R-1) \times 22 \text{ us} = 5.6 \text{ ms}$ a 256 rank: due ordini di grandezza sotto i 3.1 s misurati. Il modello a startup non spiega la crescita.
- Un modello a volume per scheda di rete ($\beta \times \text{volume off-node per nodo}$, che cresce con i rank per nodo) segue la curva misurata entro un fattore 2-3 su tutto il range: la crescita del weak scaling di pure MPI viene dal fatto che 32 rank per nodo iniettano 32 volte più traffico per NIC, non dal numero di message startup.
- La conclusione del report (l'ibrido vince perché tiene basso il numero di partecipanti) resta valida, ma il meccanismo quantitativo corretto è il volume per NIC più la contesa, non il termine $(R-1) \alpha$.

Esp. 3: continuum rank per nodo

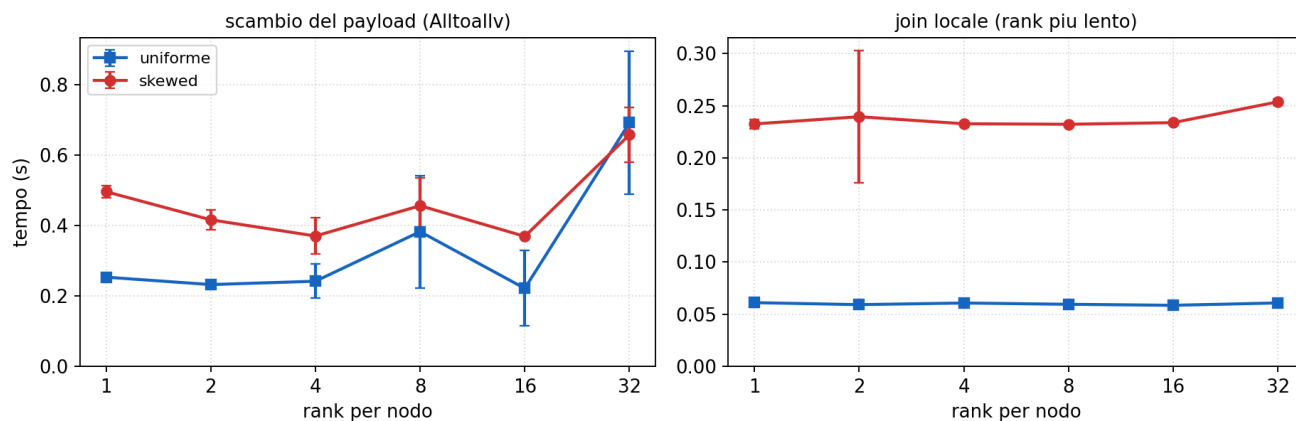


Tempo totale al variare dei rank per nodo, a 4 nodi.

- Il report confronta solo gli estremi: 32 rank/nodo (pure MPI) e 1 rank/nodo (hybrid). Qui il continuum a parità di 128 core: rank per nodo in {1, 2, 4, 8, 16, 32} con thread OpenMP complementari.
- Il minimo non è a nessuno dei due estremi: su uniforme la configurazione migliore è 16 rank x 2 thread (0.37 s contro 0.59 dell'ibrido e 0.78 del pure MPI); su skewed le configurazioni da 4 a 16 rank/nodo stanno a 0.81-0.85 s contro 1.12 e 1.41 degli estremi.
- La lettura: bastano pochi thread per parallelizzare le fasi locali dentro il rank, e fermarsi a 64 rank totali evita il regime instabile della collettiva a 128; gli estremi pagano ciascuno uno dei due costi per intero.

Esp. 3: continuum rank per nodo

Le due componenti lungo il continuum

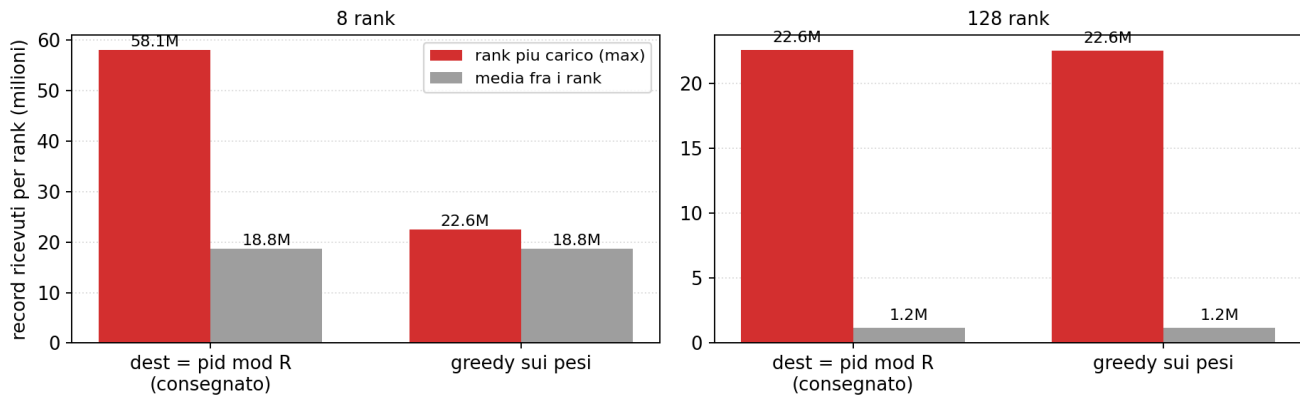


Scambio del payload e join locale lungo il continuum.

- Lo scambio del payload è piatto fino a 16 rank/nodo (64 rank totali) e salta a 32 rank/nodo (128 rank totali, 0.66-0.69 s): è la stessa soglia del microbenchmark dell'esp. 1, che si manifesta dentro la pipeline.
- Il join locale (max fra i rank) è indipendente dalla configurazione su uniforme (0.06 s) e dominato dalle partizioni hot su skewed (0.23-0.25 s ovunque): conferma che l'imbalance sotto skew non si sposta cambiando la ripartizione rank/thread.

Esp. 4: imbalance e remapping sotto skew

Volume ricevuto per rank sotto skew: a 128 rank il pavimento e la partizione indivisibile (22.6M record), nessun mapping puo scendere sotto

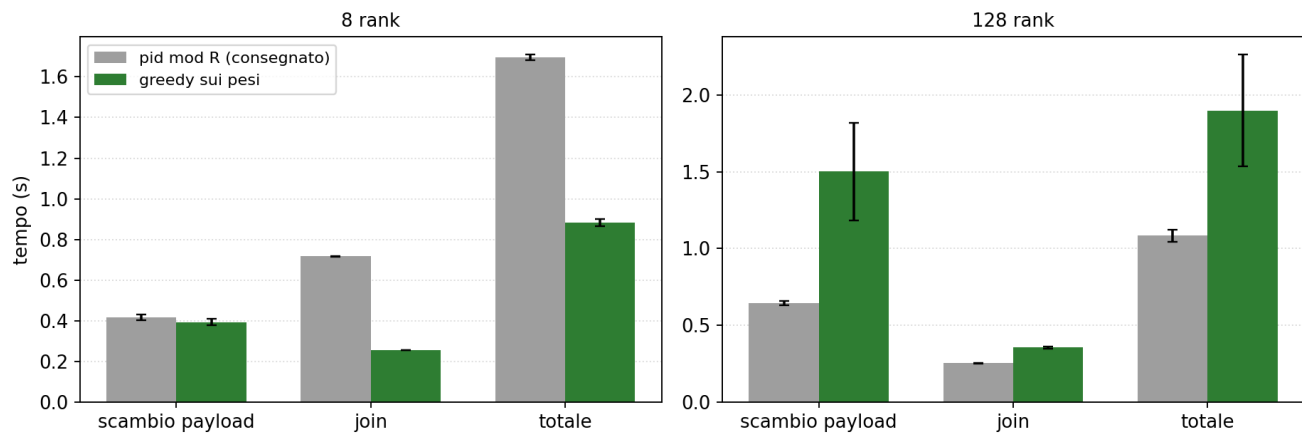


Volume ricevuto per rank: max contro media.

- Il numero che il report cita senza quantificare: sotto skew a 128 rank il rank più carico riceve 22.6M record contro una media di 1.2M (rapporto 19x). È il pavimento strutturale: la partizione hot più pesante è indivisibile e da sola vale il 15% dell'input globale (R e S hanno hot key diverse, generate da seed diversi, quindi le partizioni hot sono fino a 8: le 4 di S pesano 22.5M l'una, le 4 di R 11.3M).
- A 8 rank il mapping mod soffre di collisioni: più partizioni hot cadono sullo stesso rank (58.1M ricevuti, circa due hot di S più una di R). Il remapping greedy dai pesi globali le separa e riporta il massimo al pavimento di 22.6M.

Esp. 4: imbalance e remapping sotto skew

Effetto del remapping sui tempi (skew): grande a 8 rank, nullo o negativo a 128

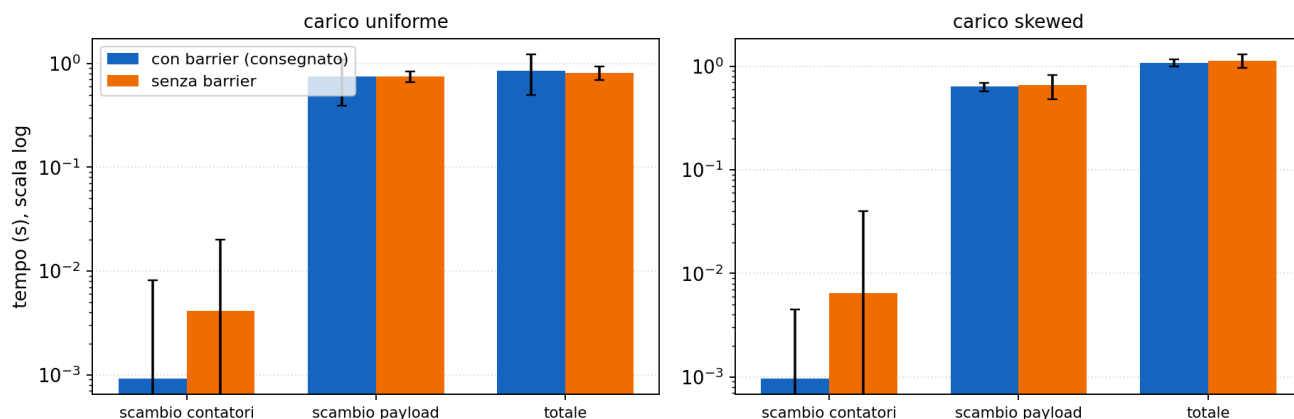


Effetto del remapping sui tempi.

- A 8 rank il greedy dimezza il totale (0.88 contro 1.70 s) tagliando il join del rank più carico da 0.72 a 0.26 s: quando l'imbalance è da collisione, il remapping consapevole dei pesi lo rimuove al costo di un Allreduce da 2-3 ms sull'istogramma globale.
- A 128 rank il greedy non guadagna nulla sul join (il pavimento è la partizione indivisibile) e peggiora lo scambio (1.50 contro 0.64 s): i send count non più regolari degradano la collettiva. Il totale peggiora del 75%.
- Questo è esattamente il confine dichiarato nel report: oltre il pavimento della partizione indivisibile serve la replicazione del build side con split del probe, non un mapping migliore.

Esp. 4: imbalance e remapping sotto skew

Metodologia di timing: MPI_Barrier davanti alle collettive (128 rank, mod)

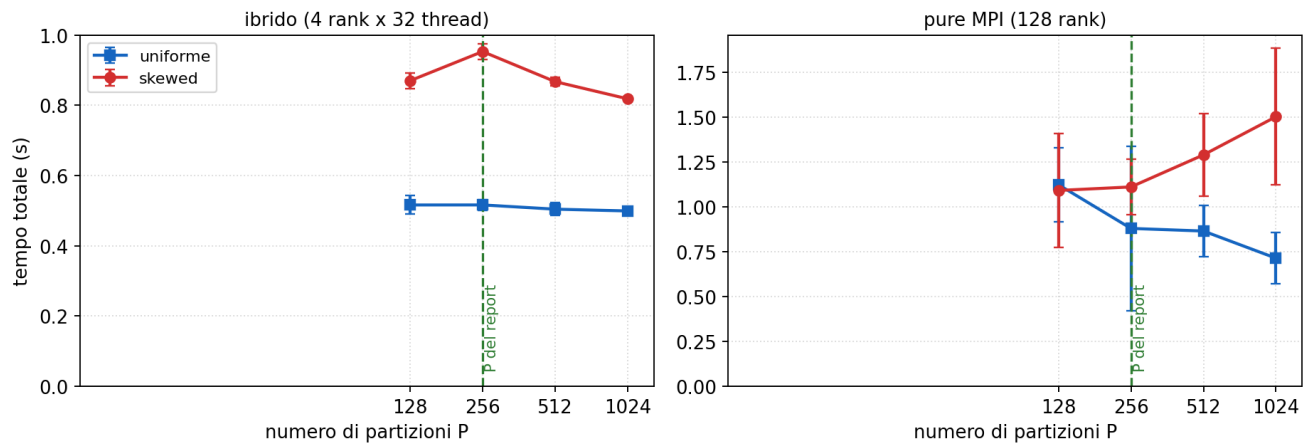


Metodologia: barrier davanti alle collettive, on/off.

- Senza i barrier lo scambio dei contatori assorbe lo skew di arrivo delle fasi locali (da 1 a 4-7 ms), mentre il payload resta dominato dal proprio costo; il totale cambia di meno del 5%.
- La scelta metodologica del report (barrier prima di ogni collettiva, riduzione max per fase) è quindi difendibile: separa il costo dello scambio dall'attesa, e il suo effetto sul totale è marginale a questi volumi.

Esp. 5: sensibilità a P

Sensibilità al numero di partizioni (4 nodi)

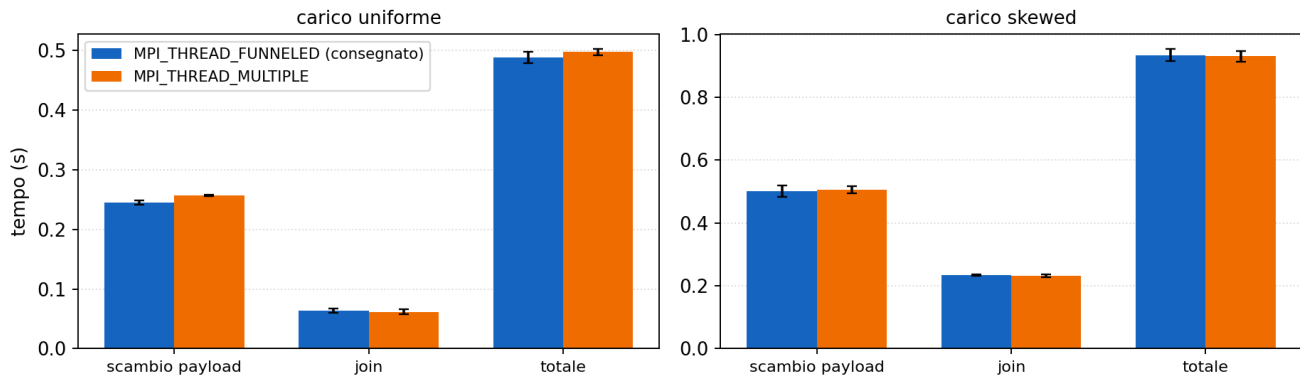


Tempo totale al variare del numero di partizioni.

- Vincolo strutturale: P deve essere multiplo dei rank, quindi il pure MPI a 128 rank ammette solo P da 128 in su.
- Ibrido: P conta poco sul totale (0.50-0.52 s su uniforme); il join migliora con P (le tabelle per partizione rientrano in cache, da 74 a 40 ms) ma lo scambio non cambia.
- Pure MPI su uniforme: P=1024 è nettamente meglio di P=128 (0.71 contro 1.12 s), sia per il join sia per lo scambio (send count più regolari). Su skewed l'ordine si inverte oltre P=256.
- P=256 del report è una scelta di mezzo ragionevole per coprire entrambe le implementazioni e i due carichi con un solo valore.

Esp. 6: livello di thread support

Livello di thread support MPI: differenza dentro il rumore
(ibrido, 4 nodi, 4 rank x 32 thread, collettive fuori dalle regioni OpenMP)



FUNNELED vs MULTIPLE sulla stessa pipeline ibrida.

- La stessa pipeline ibrida (4 rank x 32 thread) inizializzata con i due livelli: la differenza su ogni fase e sul totale è dentro il rumore (0.49 contro 0.50 s su uniforme).
- In questo codice tutte le chiamate MPI avvengono fuori dalle regioni OpenMP, quindi il locking aggiuntivo di MULTIPLE non viene mai esercitato sul percorso caldo.
- La scelta di FUNNELED resta corretta come principio (chiedere il livello minimo necessario documenta il contratto di concorrenza), ma l'argomento di costo va formulato come potenziale, non come overhead misurato.